

Архитектурные стили и оценка продуктивности слоёв

Table of contents

Архитектурные стили и оценка продуктивности слоёв	1
Чистая архитектура (Clean Architecture)	1
Продуктивность слоя	2
Альтернативные архитектурные паттерны	4
Архитектура на основе фреймворка (Rails, Django)	4
Модель акторов (Erlang/Elixir/Akka)	4
Проектирование, ориентированное на данные (Data-oriented design)	4

Архитектурные стили и оценка продуктивности слоёв

В предыдущем разделе были зафиксированы локальные правила проектирования компонентов и связей между ними. Теперь рассмотрим архитектуру на уровне всей системы: направление зависимостей, оценку продуктивности слоёв и альтернативные архитектурные подходы.

Чистая архитектура (Clean Architecture)

Чистая архитектура задаёт правило направленности зависимостей: внешние слои зависят от внутренних, но не наоборот.

Типичная структура: - **Entities / Domain**: бизнес-сущности и инварианты; - **Use Cases / Application**: сценарии применения и оркестрация бизнес-операций; - **Interface Adapters**: контроллеры, presenters, репозитории-адаптеры; - **Frameworks & Drivers**: веб-фреймворк, база данных, брокеры сообщений, внешние API.

(диаграмма опущена при переносе)

Ключевая идея: бизнес-правила должны оставаться работоспособными при смене базы данных, очереди или веб-фреймворка. Это не исключает использование фреймворков; это требует, чтобы фреймворк был деталью, а не центром архитектуры.

Связка с предыдущим материалом: - SOLID снижает связанность на уровне отдельных компонентов; - DI управляет связями между компонентами; - Clean Architecture фиксирует направление зависимостей на уровне всей системы.

Вместе они формируют практический каркас, позволяющий масштабировать кодовую базу без неконтролируемого роста архитектурного долга.

Продуктивность слоя

После фиксации принципов проектирования можно количественно оценить качество декомпозиции слоёв.

Прокси-компонент — компонент, не реализующий собственной вычислительной ответственности, а лишь делегирующий вызовы компонентам других слоёв.

Продуктивность слоя — количественная мера архитектурной самостоятельности слоя, определяемая долей компонентов, реализующих собственную вычислительную ответственность.

Формальное задание меры

Пусть: - N — общее количество компонентов в слое - P — количество прокси-компонентов в слое

Тогда коэффициент проксирования слоя определяется как:

$$K_{\text{проху}} = \frac{P}{N}$$

а продуктивность слоя — как дополняющая величина:

$$K_{\text{прод}} = 1 - K_{\text{проху}} = 1 - \frac{P}{N}$$

В стандартной литературе (например, Clean Architecture) слой считается продуктивным, если $K_{\text{прод}} \geq 0.8$, то есть не менее 80% компонентов реализуют собственную вычислительную ответственность.

Пример непродуктивного проксирующего слоя:

Рассмотрим излишний слой сервисов, большинство компонентов которого только делегируют вызовы к слою персистентности:

```
// service/user_service.go - непродуктивный слой сервисов
package service

import (
    "example/domain"
    "example/persistence"
    "strings"
)

func GetUser(id int) (*domain.User, error) {
    return persistence.LoadUser(id)
}

func SaveUser(user *domain.User) error {
```

```

    return persistence.SaveUser(user)
}

func DeleteUser(id int) error {
    return persistence.DeleteUser(id)
}

func ListUsers() ([]*domain.User, error) {
    return persistence.ListUsers()
}

func SearchUsersByName(query string) ([]*domain.User, error) {
    users, err := persistence.ListUsers()
    if err != nil {
        return nil, err
    }

    var result []*domain.User
    for _, u := range users {
        if strings.Contains(strings.ToLower(u.Name), strings.ToLower(query)) {
            result = append(result, u)
        }
    }
    return result, nil
}

```

Анализ слоя сервисов:

Компонент	Контракт	Собственная ответственность
GetUser	(int) (*User, error)	Нет — прямая делегация
SaveUser	(*User) error	Нет — прямая делегация
DeleteUser	(int) error	Нет — прямая делегация
ListUsers	() ([]*User, error)	Нет — прямая делегация
SearchUsersByName	(string) ([]*User, error)	Да — фильтрация и поиск

Продуктивность слоя: $K_{\text{prod}} = 1 - \frac{4}{5} = 0.2$ (20% — непродуктивный слой)

Такой слой не добавляет архитектурной ценности и лишь усложняет кодовую базу, увеличивая количество уровней косвенности без предоставления дополнительной функциональности.

Альтернативные архитектурные паттерны

После изучения классических подходов (слоистая архитектура, Clean Architecture, SOLID) важно понять, что **нет единственно правильного способа** организации кода. Разные задачи требуют разных подходов.

Раздел показывает альтернативные парадигмы: - **Framework-based архитектура** (Rails, Django) — фреймворк диктует структуру, вы заполняете пробелы. Быстро для типовых задач, но ограничивает гибкость. - **Actor Model** (Erlang, Akka) — вместо объектов и функций — изолированные процессы, обменивающиеся сообщениями. Идеально для высоконагруженных телекоммуникационных систем. - **Data-Oriented Design** — оптимизация под архитектуру процессора (кеш-линии, SIMD). Критично для игровых движков и симуляций, где производительность — главное.

Выбор архитектуры зависит от контекста: стартапу нужна скорость разработки (Rails), телеком-системе — отказоустойчивость (Erlang), игровому движку — производительность (DOD). Нет «лучшей» архитектуры — есть подходящая для конкретной задачи.

Архитектура на основе фреймворка (Rails, Django)

Фреймворк предполагает стандартную структуру папок, именование файлов и шаблоны проектирования (зачастую, MVC), что уменьшает количество принимаемых решений и ускоряет разработку. - “Мнения” фреймворка: Rails/Django имеют предустановленные “мнения” по работе с БД (ORM), маршрутизацией, шаблонизацией и аутентификацией.

Модель акторов (Erlang/Elixir/Akka)

- Революционная технология, созданная в Ericsson еще в 1986 году для телекоммуникационных систем с требованием “пяти девяток” доступности.
- Она на десятилетия опередила современные подходы: предоставляет
 - встроенную распределенность (кластеры из коробки),
 - философия “Let it Crash!” с автоматическим самовосстановлением
 - горячее обновление кода без остановки системы
 - невероятную параллельность (миллионы изолированных процессов)
- WhatsApp: Всего 50 инженеров обрабатывали 2 миллиарда сообщений в день на одном сервере!
- Discord

Проектирование, ориентированное на данные (Data-oriented design)

Подход, фокусирующийся на эффективной организации данных в памяти для оптимального использования кэша процессора. Используется где производительность упирается в скорость доступа к памяти. - игровые движки - обработки физики - AI - симуляции